

Handbuch & Entwickler-Leitfaden

Dokumentation zur persistenten Cross-Type ToDo-Anwendung (Java Swing & Maven)

Dieses Dokument dient sowohl als **Bedienungsanleitung (User Manual)** für Endanwender als auch als **Architektur-Leitfaden (Developer Guide)** für externe Softwareentwickler. Wenn dieses Projekt im Rahmen einer Open-Source-Veröffentlichung oder eines studentischen Portfolios publiziert wird, bietet dieses Handbuch die ideale Orientierung für kollaborative Erweiterungen und Code-Reviews.

Teil 1: Bedienungsanleitung (Anwenderhandbuch)

Die ToDo-Anwendung ermöglicht das Erstellen, Verwalten und dauerhafte Speichern von Aufgaben und Freitextnotizen über eine intuitive, im modernen Dark Mode gestaltete grafische Benutzeroberfläche.

1.1 Starten der Anwendung

Die Anwendung wird entweder direkt über eine integrierte Entwicklungsumgebung (IDE) durch Ausführen der Klasse `app.Main` oder als eigenständige ausführbare Binärdatei gestartet:

```
java -jar ToDoApp-1.0-SNAPSHOT-jar-with-dependencies.jar
```

1.2 Die Benutzeroberfläche im Überblick

Das Anwendungsfenster ist in eine funktionale Zwei-Seiten-Struktur (Split-Pane-Layout) unterteilt:

- **Linke Navigationsleiste:** Hier werden alle angelegten Listen chronologisch untereinander aufgelistet. Am unteren Ende befinden sich drei Steuerelemente: *"Neue Text-Liste"*, *"Neue Checkbox-Liste"* und der rote Button *"Ausgewählte Liste löschen"*.
- **Rechter Detailbereich:** Dieser Bereich passt sich dynamisch an. Wenn links keine Liste selektiert ist, erscheint ein Platzhaltertext. Sobald eine Liste ausgewählt wird, lädt das System das entsprechende Interface (Texteditor oder interaktive Checkbox-Zeilen).

1.3 Bedienung der Listentypen

Die Anwendung unterscheidet strikt zwischen zwei funktionalen Modulen:

1. **Text-Listen (Fließtext-Notizen):** Ideal für unstrukturierte Notizen, Protokolle oder Gedankenstützen. Das Textfeld speichert jede Eingabe vollautomatisch und verzögerungsfrei im Hintergrund, während der Anwender tippt.

2. **Checkbox-Listen (Aufgabenlisten):** Konzipiert für strukturierte Todos. Über das untere Eingabefeld und den Button "Hinzufügen" (oder das Drücken der Enter-Taste) werden neue Aufgaben erfasst.
- **Intelligente Sortierung:** Wird eine Aufgabe per Klick abgehakt, wird der Text visuell durchgestrichen, ausgegraut und die gesamte Zeile wandert automatisch an das Ende der Liste, um offene Aufgaben im Fokus zu halten.
 - **Einzelne Einträge löschen:** Rechts neben jeder Aufgabe befindet sich ein dezentes rotes Kreuz-Symbol (✖). Ein Klick darauf entfernt die jeweilige Aufgabe unwiderruflich.

Sicheres Löschen von Listen

Um versehentlichen Datenverlust zu vermeiden, öffnet das Klicken auf "Ausgewählte Liste löschen" ein modales Bestätigungsfenster. Erst nach expliziter Zustimmung des Nutzers wird die gesamte Liste aus dem Speicher entfernt.

1.4 Datenspeicherung (Persistenz)

Die Anwendung verfügt über ein nahtloses Speicherkonzept. Es ist kein manueller "Speichern"-Button notwendig. Beim ordnungsgemäßen Schließen des Fensters über das rote System-Kreuz fängt ein dedizierter Listener das Event ab, liest den aktuellen UI-Zustand aus und serialisiert die Daten in eine lokale Datei namens `daten.json` im Projekt-Hauptverzeichnis. Beim nächsten Programmstart wird diese Datei automatisch wieder eingelesen.

Teil 2: Technische Dokumentation (Architektur & Code)

Für Entwickler, die den Funktionsumfang erweitern oder Optimierungen am Quelltext vornehmen möchten, ist die Einhaltung der Clean-Code-Richtlinien und der MVC-ähnlichen Architektur zwingend erforderlich.

2.1 Die Paket-Struktur (Package-Design)

Das Projekt ist in vier klar abgegrenzte Sub-Packages unterteilt, um eine hohe Kohäsion und lose Kopplung zu gewährleisten:

Package Name	Verantwortlichkeit und Inhalt
<code>app</code>	Einstiegspunkt des Programms. Enthält die Klasse <code>Main</code> , die den Swing-Thread (EDT) sicher initialisiert.
<code>model</code>	Das reine Domänenmodell (Datenklassen). Frei von GUI-Logik. Enthält <code>ToDoList</code> , <code>TextToDoList</code> , <code>CheckboxToDoList</code> und <code>ToDoItem</code> .
<code>ui</code>	Präsentationsschicht. Enthält das Kern-Interface <code>MainFrame</code> , die UI-Konstanten (Dark-Mode-Farbe) und Event-Listener.
<code>storage</code>	Infrastrukturschicht für Daten-Persistenz. Beinhaltet den <code>StorageManager</code> sowie den GSON-spezifischen <code>ToDoListAdapter</code> .

2.2 Polymorphe Datenstrukturen und JSON-Serialisierung

Die größte architektonische Besonderheit liegt in der polymorphen Speicherung der Listen. Da die abstrakte Oberklasse `ToDoList` unterschiedliche Implementierungen besitzt (Text vs. Checkbox), reicht ein Standard-JSON-Parser nicht aus. Beim Konvertieren in Textform würden die spezifischen Attribute der Unterklassen verloren gehen (Object Slicing).

Gelöst wurde dies über die Implementierung der Schnittstellen `JsonSerializer<ToDoList>` und `JsonDeserializer<ToDoList>` in der Klasse `ToDoListAdapter`. Beim Speichern wird ein künstliches Meta-Attribut namens `"type"` in das JSON-Objekt injiziert:

```
{
  "type": "CheckboxToDoList",
  "properties": {
    "items": [ ... ],
    "title": "Einkauf"
  }
}
```

Beim Einlesen wertet der Adapter dieses Feld aus und instanziiert per Reflection die exakt korrekte Java-Unterklasse. Entwickler müssen bei Änderungen am Datenmodell sicherstellen, dass dieser Adapter synchron gehalten wird.

Teil 3: Erweiterungs-Leitfaden (Anlaufstellen für Code-Änderungen)

Wenn du das Projekt erweitern, Code-Beiträge leisten oder die App modifizieren möchtest, findest du hier die exakten Einstiegspunkte für die häufigsten Anpassungswünsche:

3.1 Einen neuen Listentyp hinzufügen

Szenario: Du möchtest neben Text und Checkboxes einen dritten Typ einführen (z. B. eine "Termin-Liste" mit Datumsauswahl).

1. Erstelle im Package `model` eine neue Klasse, die von der abstrakten Klasse `ToDoList` erbt (z. B. `class EventToDoList extends ToDoList`).
2. Erweitere die Klasse `storage.ToDoListAdapter` : Füge in der Methode `deserialize` eine weitere `else if`-Bedingung hinzu, um den neuen Typ-String abzufangen und zuzuordnen.
3. Erweitere die Klasse `ui.MainFrame` :
 - Füge im linken Panel einen dritten Erstellungs-Button hinzu.
 - Erweitere die Verzweigung innerhalb der Methode `updateRightPanel` um ein entsprechendes GUI-Layout für deinen neuen Listentyp (z. B. unter Verwendung von Datumskomponenten).

3.2 Das visuelle Design anpassen (Theme / Farbschema)

Szenario: Der Dark Mode soll durch ein helles Design ("Light Mode") ersetzt werden oder die Akzentfarben der Buttons sollen sich ändern.

Alle visuellen Design-Entscheidungen sind zentral als Konstanten im Kopf der Klasse `ui.MainFrame` hinterlegt. Es ist kein Suchen im gesamten Code notwendig. Verändere dort einfach die statischen RGB-Werte der Farb-Konstanten:

```
private final Color ACCENT_COLOR = new Color(88, 166, 255); // Akzent-Blau
private final Color DANGER_COLOR = new Color(231, 76, 60); // Lösch-Rot
private final Color BG_LEFT_COLOR = new Color(33, 33, 33); // Hintergrund Links
```

3.3 Eine Suchfunktion oder Filterleiste integrieren

Szenario: Bei vielen ToDo-Listen soll eine Suchzeile oberhalb der Navigation das Filtern ermöglichen.

Die richtige Anlaufstelle hierfür ist die Methode `initUI()` in der `MainFrame.java`. Du müsstest ein `TextField` oberhalb der `JList` im `leftPanel` platzieren. Über einen angehängten `DocumentListener` am Suchfeld kannst du bei jeder Tastatureingabe das `ListModel` filtern, indem du Elemente temporär entfernst oder aus einer Master-Liste neu hinzulädst.

Wichtiger Hinweis für Code-Beiträge (Pull Requests)

Wenn du Modifikationen am Code vornimmst, führe vor jedem Git-Push oder Pull Request im Terminal den Befehl `mvn clean package` aus. Dadurch wird sichergestellt, dass das Projekt fehlerfrei kompiliert, alle Abhängigkeiten korrekt aufgelöst werden und die ausführbare JAR-Datei im Ordner `target/` ohne Warnungen generiert werden kann. Unvollständiger Code, der den Build-Prozess blockiert, darf nicht in den produktiven Master-Branch gemergt werden.